

Chatfat Messaging

Secure persistent group chat with performance-based dynamic load balancing

Viva and assignment report

Public endpoint: <https://10.1.75.51:3201>

The deployed certificate is self-signed; accept it once in a browser before using WebCrypto features.

Scope and evidence

This report describes the implementation in the repository and uses checked-in experiment artifacts. The threshold sweep uses PostgreSQL-backed traffic; the final deployment uses the same shared persistence model. Benchmark-only runs using DATABASE_URL=none are clearly separated from the persistent results.

1. Assignment mapping and architecture

The assignment requires three backend copies, one client-facing load balancer, performance-aware routing, health eviction, shared persistent data, duplicate-safe messages, /message and /feed, and a variable load generator. All required routes remain in the Node application and are proxied by the load balancer.

Deployment topology

Sys1: Go load balancer, public HTTPS listener on port 3201.

Sys2: backend-1 at 172.17.0.3:4202.

Sys3: backend-2 at 172.17.0.4:4203.

Sys4: backend-3 at 172.17.0.5:3204 and PostgreSQL on port 5000.

Request path

Browser or load generator -> HTTPS listener -> Go backend selector -> healthy Node backend -> shared PostgreSQL -> response. WebSocket upgrades follow the same proxy path. The browser never needs to know which backend handled a request.

Technologies

Layer	Technology	Responsibility
Frontend	HTML, JavaScript, WebCrypto	Authentication, encrypted rooms, signatures, logout
Realtime	Node.js, raw WebSocket (ws)	Chat transport and room events
Auth	Better Auth	Signup, sessions, sign-out, trusted origins
Persistence	PostgreSQL via pg	Shared messages, users and duplicate-safe writes
Load balancer	Go net/http	P2C routing, metrics, TLS, health and proxying
Recovery	Shell watchdogs	Restart failed backends and LB processes

2. Dynamic load-balancing design

The load balancer uses Power of Two Choices (P2C), not fixed round-robin. For each request it samples two healthy candidates and chooses the lower score. The score combines observed service latency (EWMA), event-loop lag, and in-flight work. This makes a slow or saturated backend less attractive while preserving distribution.

Current deployed controls

Setting	Value	Purpose
Selection	P2C	Performance-based candidate choice
Load threshold	200 ms	Operational switching threshold
Health interval	250 ms	Frequent liveness measurement
Health timeout	600 ms	Bound probe wait
Unhealthy threshold	3 failures	Hysteresis against transient overload
Healthy threshold	1 success	Fast restoration
Backend timeout	45 s	Protect client and proxy resources

Health and failure handling

Failed probes mark a backend unhealthy after the configured consecutive-failure count. It is removed from selection, while successful probes restore it. Connection-level proxy failures can evict a backend; overload timeouts return 504 and remain a health-loop decision rather than immediately removing every busy backend. This avoids the collapse where a saturated single backend is mistaken for a dead backend.

Watchdogs run on Sys1-Sys4, check process and health status, and restart after repeated failures. PID and duplicate-start safeguards prevent an automatic recovery loop from creating competing listeners.

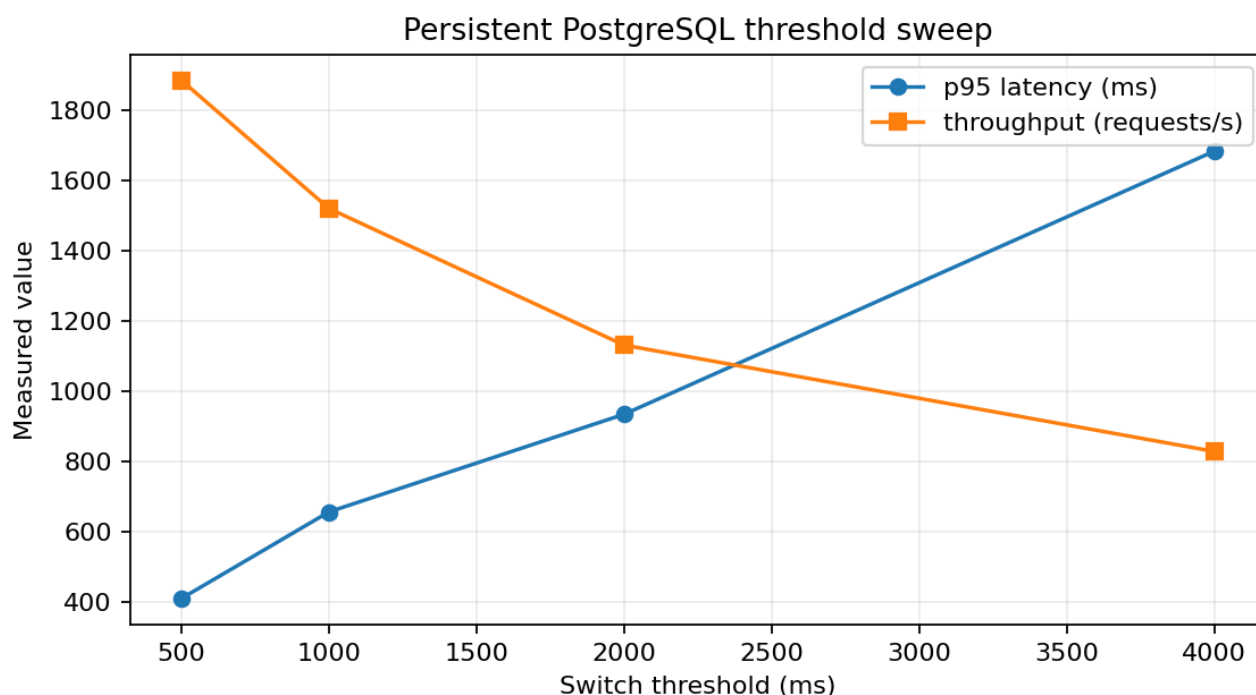


Figure 1. PostgreSQL-backed threshold sweep. Lower thresholds provided lower measured tail latency in this run; 200 ms is retained as the deployed operational setting.

3. Persistence, security and API behavior

All three backends use the same PostgreSQL URL on Sys4. Message insertion is idempotent: each message has a unique identifier and the database uniqueness constraint makes retries harmless. Duplicate insert attempts do not create a second chat record. The repository also supports batch and feed-key operations used by the feed warmer.

The exact assignment routes are preserved:

Route	Behavior
/message	Accepts client-name and msg, persists a message, returns its result
/feed	Returns persisted messages in feed order

Security features include Better Auth sessions, HTTPS at the public boundary, AES-GCM room encryption, PBKDF2-derived room keys, ECDSA P-256 message signatures, and a shared MASTER_KEY for encrypted data at rest. The Node 18 runtime receives a WebCrypto compatibility bridge; Better Auth ESM modules are loaded dynamically.

Important operational distinction

DATABASE_URL=none is acceptable only for isolated CPU/HTTP benchmarking. The final deployment must use the shared PostgreSQL URL and the original MASTER_KEY; otherwise data is not shared and existing encrypted data cannot be read.

4. Load-generation method and measured results

The repository includes Go load generators for benchmark traffic and chat-shaped traffic. They support variable users, random message sizes, variable intervals, concurrency, timeouts, warm-up, latency percentiles, backend distribution and utilization snapshots.

Threshold sweep (persistent PostgreSQL)

Threshold	Requests	p50 ms	p95 ms	p99 ms	Throughput	Dropout
500	3864	11.96	409.39	460.56	1884.55	0.00%
1000	3876	10.63	655.76	829.95	1519.61	0.00%
2000	3916	15.13	933.70	1329.42	1130.63	0.00%
4000	3892	13.45	1682.55	2129.48	827.96	0.00%

The 500 ms candidate had the best measured p95 (409.39 ms) and throughput (1884.55 requests/s) among these persistent candidates, with zero dropout. The deployed 200 ms threshold is a more responsive operational policy; the sweep demonstrates why threshold selection must balance tail latency, routing stability and workload shape.

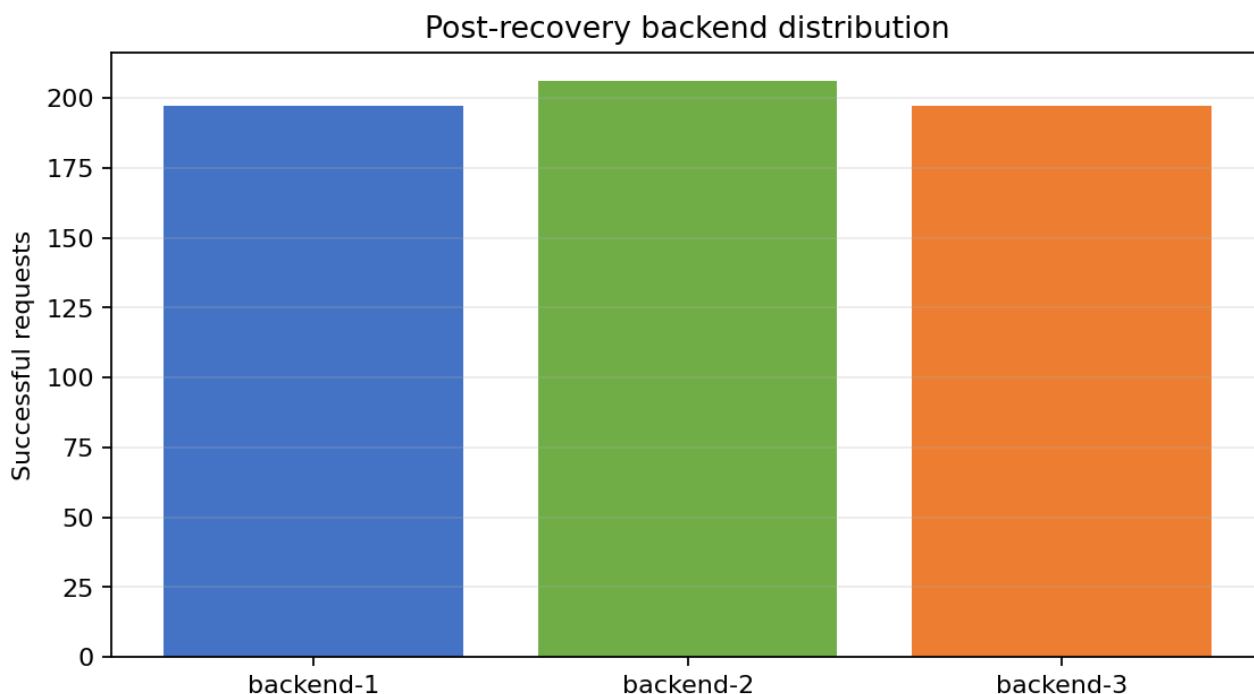


Figure 2. Recovery verification: 600 successful requests after backend recovery were distributed 197/206/197.

Recovery result

The post-recovery artifact recorded 600 successes, 0 failures, 1642.65 requests/s, p50 14.62 ms and p95 36.34 ms. All responses were HTTP 200.

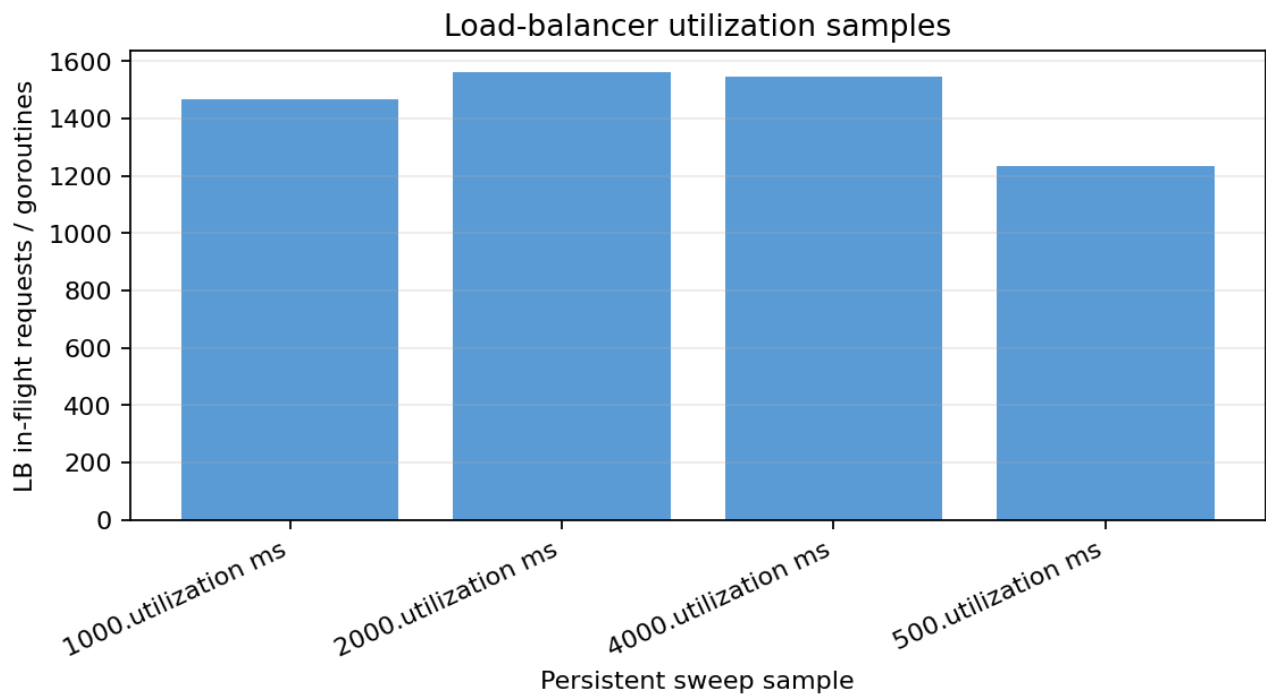


Figure 3. Load-balancer utilization samples recorded during persistent sweeps. The checked-in capture reports LB in-flight/goroutine counts and backend score notes; host CPU fields were zero in that capture, so no host CPU percentage is claimed here.

5. Troubleshooting and operations

HTTP sent to HTTPS listener: use `https://10.1.75.51:3201`, not `http://`. The message means the TLS listener received the wrong protocol and is not an application crash.

Better Auth ESM error: use `dynamic import` for `better-auth` and `better-auth/node`; Node 18 also needs the `node:crypto` WebCrypto bridge.

WebCrypto unavailable: access the browser through HTTPS and accept the self-signed certificate warning.

Backend DOWN / EOF: ensure the backend is plain HTTP if the LB health URL is `http://`; do not accidentally enable backend TLS without changing the probe URL.

Address already in use: inspect listeners with `ss -ltnp` and terminate the specific stale PID; avoid starting a second watchdog or LB process.

Database startup error: verify PostgreSQL port 5000, credentials, firewall/listen addresses and the same `DATABASE_URL` on all three backends.

Feed warmer warning: repository implementations must expose `feedKeys`, `byIds` and `saveMany` consistently.

Useful live checks

```
curl -k https://10.1.75.51:3201/healthz
curl -k https://10.1.75.51:3201/lb/status
curl -k https://10.1.75.51:3201/lb/metrics
tail -f /tmp/lb.log
tail -f /tmp/backend-1.log
```

Strengths and limitations

Strengths: real persistence, idempotent writes, encrypted rooms, authenticated sessions, adaptive routing, health hysteresis, automatic restart, metrics, and reproducible load tools.

Limitations: the lab uses a self-signed certificate; the database is a single shared PostgreSQL point of failure; and the checked-in utilization snapshot is LB-centric rather than a synchronized four-host CPU trace. A production deployment should add trusted certificates, PostgreSQL replication/failover and centralized metrics.

6. Viva questions and model answers

Why is P2C better than round-robin here?

Round-robin ignores current latency and queue depth. P2C samples two healthy servers and uses the lower live score, so slow or saturated servers receive less traffic.

What prevents a retry from duplicating a message?

The message ID is unique and the PostgreSQL constraint makes insertion idempotent. A repeated request returns the existing logical message rather than adding a row.

Why not evict on every timeout?

A busy single-threaded Node backend can timeout a probe while still alive. Immediate eviction can remove the last backend and create a false outage. Consecutive health failures and hysteresis separate overload from actual death.

What happens if Sys3 crashes?

Health probes mark it unhealthy after three failures, routing stops there. The watchdog restarts the process; one successful probe restores it without restarting the other servers.

Why does the browser need HTTPS?

WebCrypto is unavailable in the ordinary HTTP IP-address context used by the lab. HTTPS provides the secure context required for signing and encryption APIs.

What is the single biggest availability risk?

The shared PostgreSQL instance on Sys4. Backend redundancy does not remove a database failure; production mitigation would be replication, failover and backups.

How would you choose a threshold for a new workload?

Run the same workload over a sweep, compare p95/p99, throughput and dropout, then select the lowest stable tail latency without routing flapping. Recheck after deployment.

How do you prove traffic reached all backends?

Use X-Backend or per-backend counters in /lb/status and compare them with health state and load-generator output.

What if all backends are overloaded but healthy?

P2C still chooses the least-loaded candidate, timeouts are bounded, metrics expose the saturation, and the next step is capacity or workload scaling rather than falsely evicting every server.

Conclusion

The final system satisfies the assignment's routing, persistence, dynamic balancing, health detection and recovery requirements while retaining the secure group-chat behavior. The public submission endpoint is <https://10.1.75.51:3201>.